

UCS503 – Software Engineering Project
Academic Year 2026–2027

VendorFlow

A Full-Stack SaaS Platform for Local Grocery & Pharmacy
Digitization

Inventory, Online Orders, Delivery Coordination & Sales Analytics

Submitted by:

1024240017 Mannat Saini

1024240018 Kunal

1024240031 Pranjali

B.E. Third Year, Computer Science & Engineering

Submitted to: Dr. Jeelani Asif

Dr. [Instructor Name]

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Data Model	5
2.3	Implementation Details	5
2.3.1	Module 1: Order Processing & Delivery Workflow	5
2.3.2	Module 2: Smart Vendor Routing & Reliability Scoring	5
2.3.3	Module 3: Role-Based Portals	9
3	Results and Evaluation	10
3.1	Working Prototype	10
3.2	Testing Methodology	10
3.3	Performance Metrics	11
3.4	Analysis	11
4	Conclusion	14
4.1	Summary of Work	14
4.2	Key Findings	14
4.3	Future Work and Recommendations	14
4.4	Final Remarks	15

Chapter 1

Introduction

1.1 Background and Context

Local grocery stores and pharmacies largely operate without any shared digital infrastructure: each business that wants an online presence must build and maintain its own website or app, manage its own delivery network, and track inventory by hand. This duplication of effort raises the barrier to going online for small, independent vendors and leaves customers with no single place to discover and order from the stores near them.

VendorFlow addresses this gap as a shared, multi-tenant SaaS platform: local grocery and pharmacy businesses join as vendors, list their inventory, and receive orders through a common marketplace, while customers browse products across every approved vendor in their area, place orders, and track deliveries end to end. A dedicated delivery-partner role closes the loop between order placement and doorstep delivery, and an admin role governs vendor onboarding and platform-wide oversight.

1.1.1 Problem Statement

Independent grocery and pharmacy vendors lack an affordable, shared way to sell online, coordinate delivery, and manage inventory, while customers lack a single platform to discover and order from multiple local stores at once. Pharmacy orders add a further requirement that most generic e-commerce platforms do not handle: prescription upload and verification before an order can be fulfilled. VendorFlow resolves this by providing one role-based platform serving customers, vendors, delivery partners, and admins, with smart vendor routing and prescription verification built in.

1.2 Project Objectives

The project's core objectives for the current build (v1 MVP) are:

1. **Role-based multi-portal access:** Deliver four dedicated portals (customer, vendor, delivery partner, admin) behind a single JWT-authenticated, role-based access control layer.
2. **Vendor onboarding & approval:** Support vendor registration with an admin approval workflow and a computed reliability score used later for order routing.

3. **Smart vendor routing:** Route each incoming order to a vendor using a combination of distance, stock availability, and reliability score, with automatic re-routing on rejection.
4. **Prescription-aware ordering:** Allow customers to upload a prescription at checkout for pharmacy items, and hold the order for admin verification before it proceeds.
5. **Order & delivery lifecycle:** Track an order from cart through checkout, vendor acceptance, delivery assignment, and final delivery, with real-time status updates pushed to all relevant portals.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

VendorFlow is built as a small set of cooperating services rather than a single monolith, so that each layer can use the language best suited to its workload. Figure 2.1 illustrates the high-level architecture.

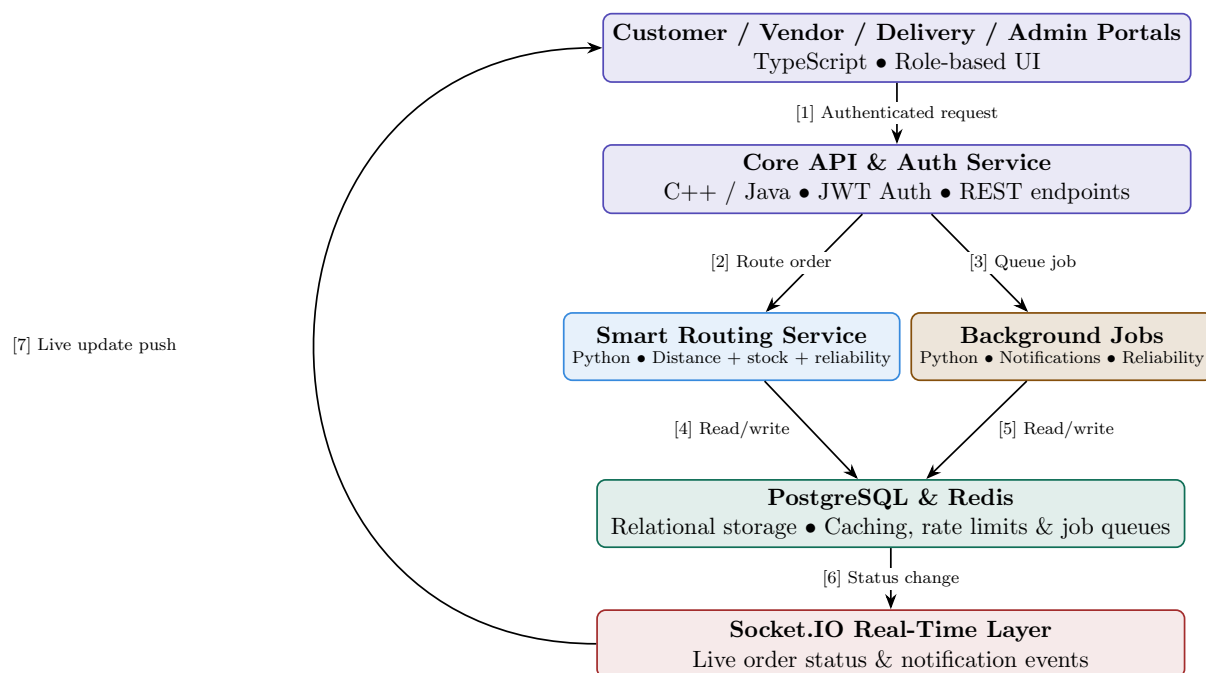


Figure 2.1: VendorFlow High-Level Multi-Service Architecture

2.1.2 Technical Stack

The implementation stack is split by workload rather than by convenience: each language handles the part of the system it is best suited for.

C++ and Java together handle the core API, authentication, and order/inventory business logic — C++ for performance-critical paths, Java for the broader service layer.

Layer	Technology	Purpose
Core backend	C++, Java	High-performance API and business logic layer
Routing & jobs	Python	Smart vendor routing, reliability scoring, background job workers
Frontend	TypeScript	Customer, vendor, delivery partner & admin portals
Primary database	PostgreSQL	Relational storage for users, orders, inventory, wallets
Cache & queues	Redis	Caching, rate limiting, background job queues
Real-time layer	Socket.IO	Live order status & notification updates

Table 2.1: VendorFlow Technology Stack

Python runs as a separate service for smart vendor routing, reliability scoring, and background job workers, tasks that change often and benefit from a faster iteration cycle. TypeScript powers all four frontend portals, sharing a single type-safe API contract with the backend. PostgreSQL is the single source of truth for relational data, with Redis in front of it for caching, rate limiting, and job queues; Socket.IO pushes order-status and notification events to connected clients in real time.

2.2 Data Model

The schema is presented at three levels of detail, moving from a high-level conceptual map down to the exact columns and data types used in the database.

2.3 Implementation Details

2.3.1 Module 1: Order Processing & Delivery Workflow

The order lifecycle is modeled as a cross-functional activity flow spanning four swimlanes — customer, VendorFlow system, vendor, and delivery partner (Figure 2.5). A customer’s checkout request is authenticated and handed to the Python smart-routing service, which selects a vendor by distance, stock, and reliability. If a vendor declines, the order is re-routed and that vendor’s reliability score is lowered. Once a vendor accepts, inventory is updated and a delivery task is published to the next available delivery partner via a Redis queue, with the same accept/re-route pattern used for delivery assignment.

2.3.2 Module 2: Smart Vendor Routing & Reliability Scoring

The routing service is implemented as a standalone Python component so that its matching logic can be iterated on independently of the core API. It scores candidate vendors on three factors — proximity to the customer, current stock of the requested items, and

Figure 1 — Conceptual ER Diagram (Level 1)

Entities and how they relate — no keys or attributes, the high-level map of the system.

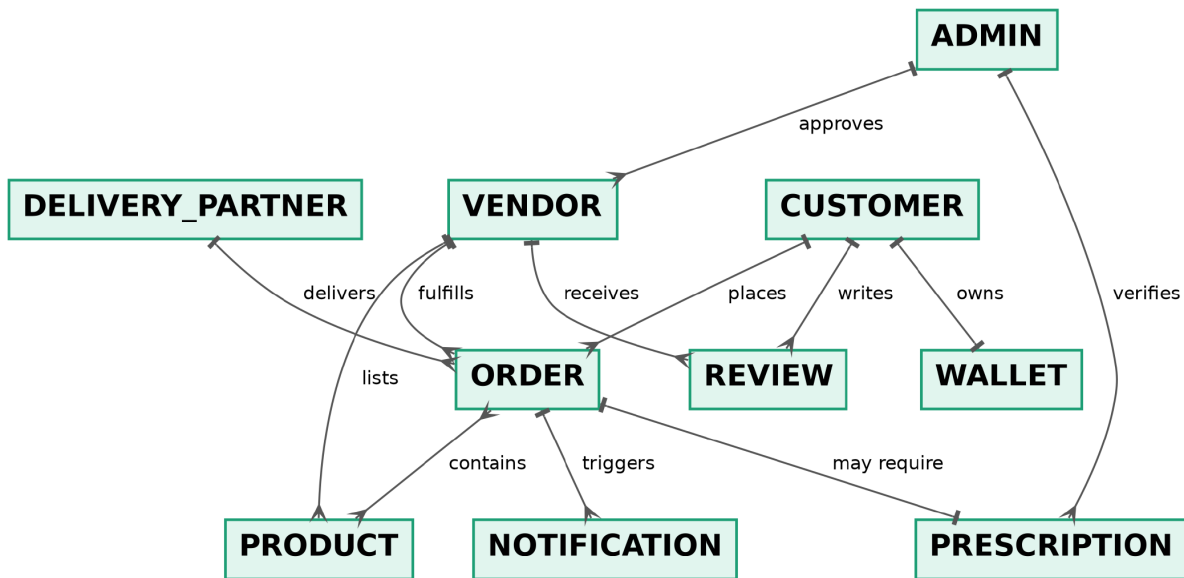


Figure 2.2: Conceptual ER Diagram (Level 1) — entities and how they relate, with no keys or attributes.

Figure 2 — Logical ER Diagram (Level 2)

Adds primary/foreign keys and the junction tables (cart, order items, delivery assignment) needed to implement the relationships.

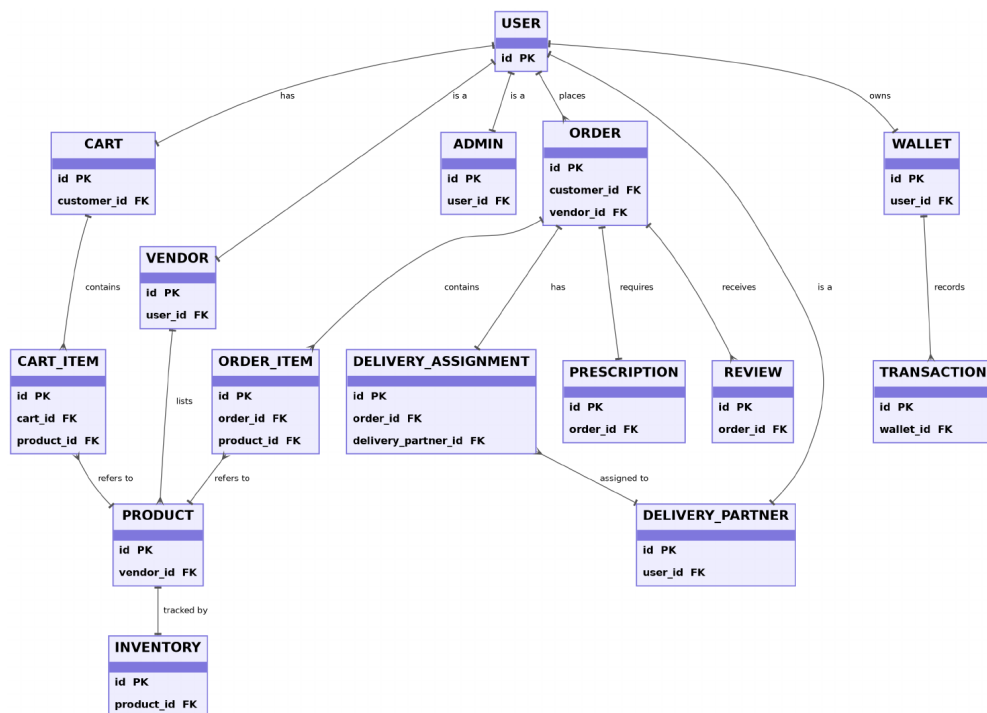


Figure 2.3: Logical ER Diagram (Level 2) — adds primary/foreign keys and the junction tables (cart items, order items, delivery assignment) needed to implement the relationships.

Figure 3 — Physical ER Diagram (Level 3)

Full column definitions with data types — the schema ready to translate into CREATE TABLE statements.

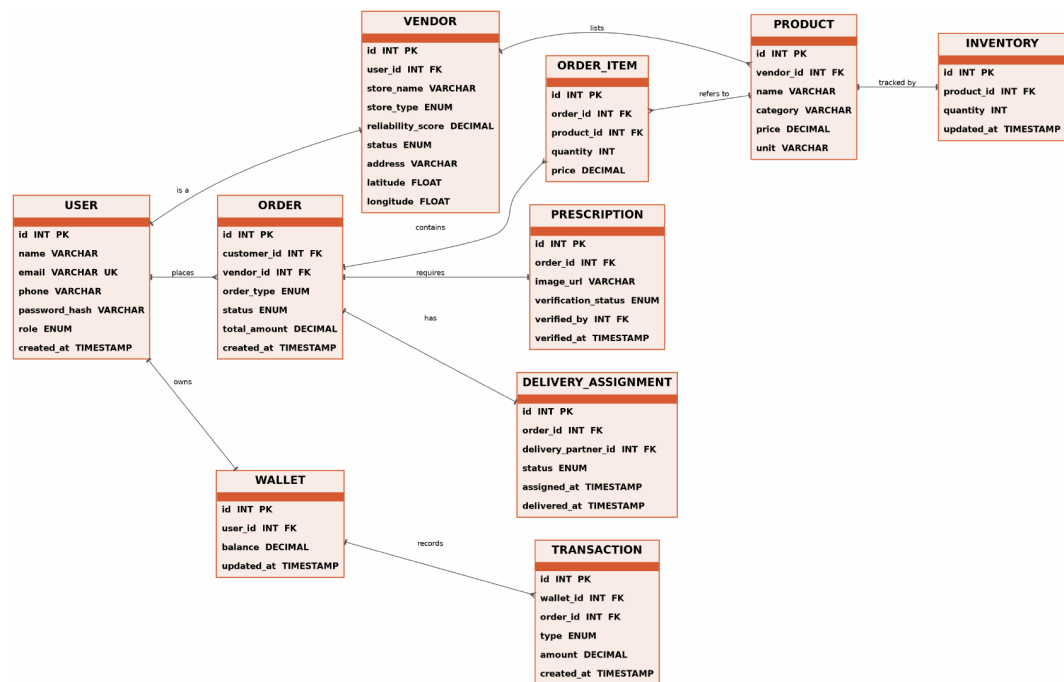


Figure 2.4: Physical ER Diagram (Level 3) — full column definitions with data types, ready to translate into CREATE TABLE statements.

Entity	Purpose
User	Base account for every person on the platform, with a role (customer / vendor / delivery / admin)
Vendor	A store's profile — type, location, approval status, reliability score
Product / Inventory	Items a vendor sells and the stock level behind each one
Cart / Cart Item	A customer's in-progress selection before checkout
Order / Order Item	A placed order and the line items within it
Prescription	Uploaded pharmacy prescription, pending or verified by an admin
Delivery Assignment	Links an order to the delivery partner fulfilling it, with live status
Wallet / Transaction	A user's running balance and the credit/debit history behind it
Review	A customer's rating and comment tied to a completed order

Table 2.2: Core Entities and Their Purpose

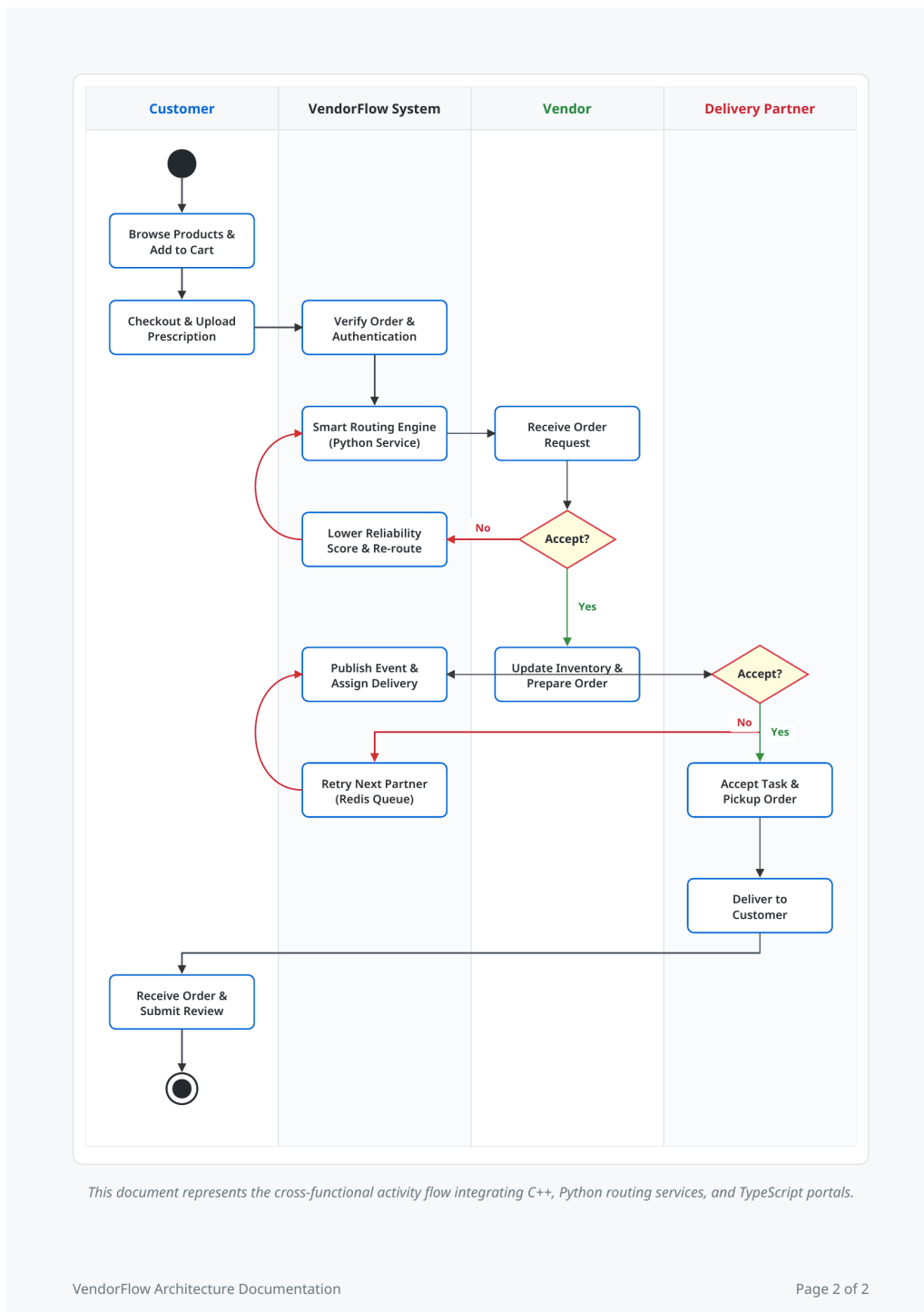


Figure 2.5: Order Processing & Delivery Activity Diagram, spanning the customer, VendorFlow system, vendor, and delivery-partner swimlanes.

a running reliability score derived from past order-acceptance and delivery outcomes — and re-scores automatically whenever a vendor rejects an order, so that unreliable vendors are progressively deprioritized without manual intervention.

2.3.3 Module 3: Role-Based Portals

All four portals share a single TypeScript codebase and API contract, with access gated by JWT-encoded roles rather than separate deployments. The customer portal supports product discovery and cart checkout across every approved vendor; the vendor portal exposes store profile, approval status, and product/inventory management; the admin portal handles the vendor approval queue and reliability oversight. Screens from the current build are shown in Section 3.1.

Chapter 3

Results and Evaluation

3.1 Working Prototype

Screens from the current build of VendorFlow cover authentication, the customer shopping flow, the vendor inventory panel, and the admin approval dashboard. Each screen maps directly back to a use case from the platform’s core feature set (Table 3.1).

Figure	Screen	Demonstrates
3.1	Login & authentication	Role-based sign-in and JWT-backed session handling
3.2	Customer — Browse Products	Product discovery across multiple approved vendors, with cart
3.3	Vendor — Dashboard	Store profile, approval status, and product/inventory management
3.4	Admin — Dashboard	Vendor approval queue and reliability-score oversight

Table 3.1: Prototype Screens and the Use Cases They Demonstrate

3.2 Testing Methodology

Testing follows a structured, multi-tier evaluation approach:

1. **Unit Testing:** Individual testing of routing-score calculations, reliability updates, and API request validation.
2. **Integration Testing:** End-to-end testing of the checkout, order-acceptance, and delivery-assignment flows across the C++/Java API and the Python routing service.
3. **Scenario Validation:** Manual walkthroughs of the three core journeys — customer checkout, vendor order acceptance, and admin vendor approval — against the demo data set (QuickMart and FreshStore vendors).

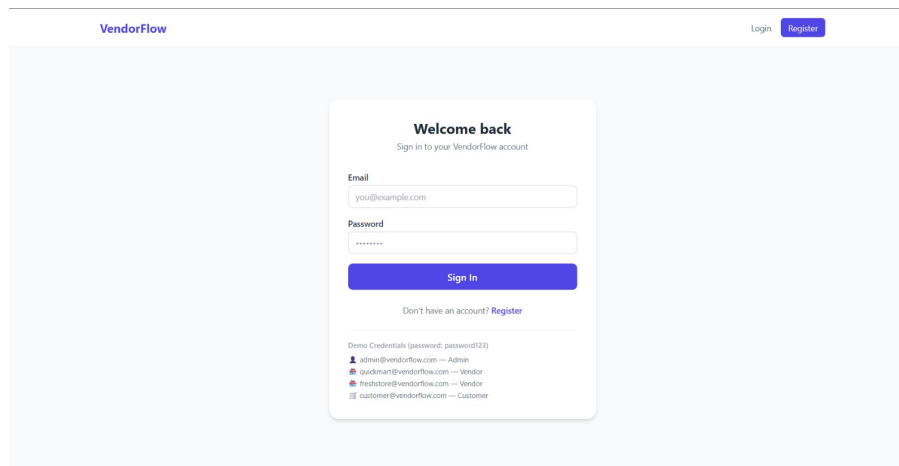


Figure 3.1: Login & Authentication — the same login screen routes customers, vendors, delivery partners, and admins to their respective portals.

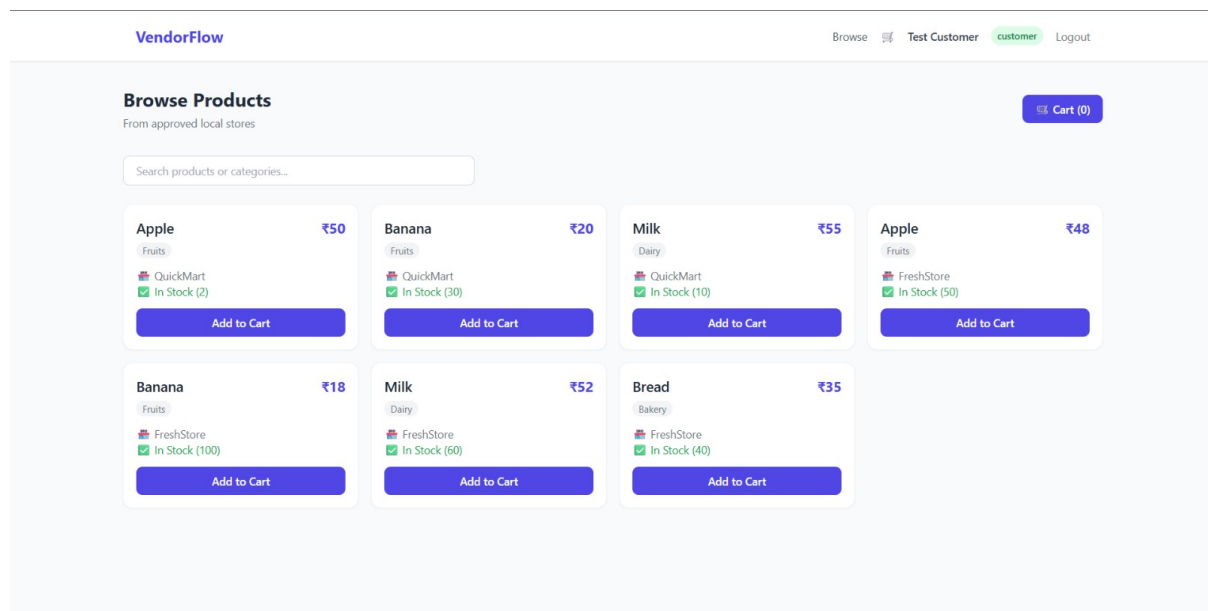


Figure 3.2: Customer Portal — Browse Products, showing search across approved local stores and cart access.

3.3 Performance Metrics

Table 3.2 summarizes target versus achieved engineering metrics for the current prototype. *Values below are placeholders — replace them with your team’s actual measured results before submission.*

3.4 Analysis

The prototype currently demonstrates the full happy-path journey across all four roles: a customer can browse products from multiple approved vendors, check out, and have the order routed to a vendor; the vendor can manage inventory and accept the order; and the admin can review and approve vendor applications with visibility into reliability scores.

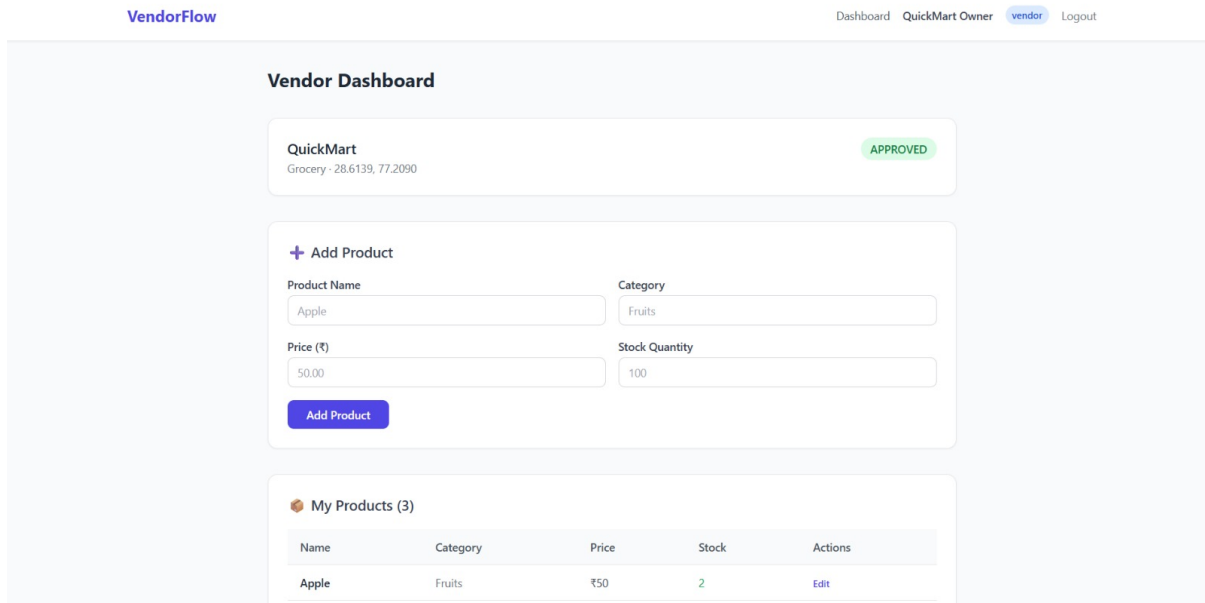


Figure 3.3: Vendor Portal — Dashboard, showing store profile, approval status, and product catalog management.

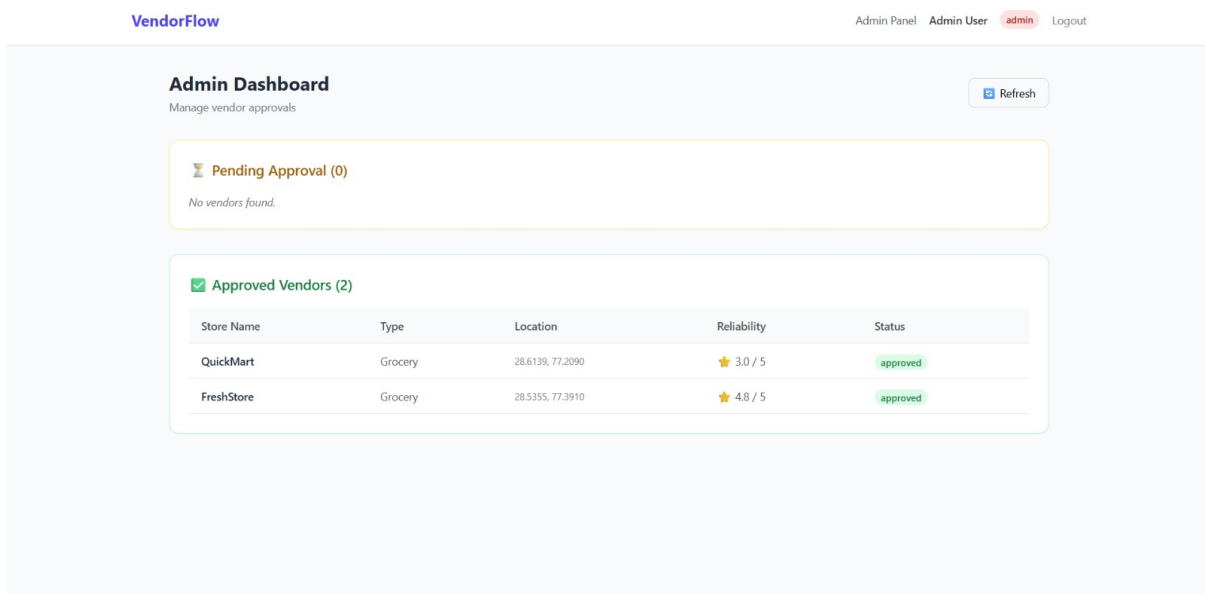


Figure 3.4: Admin Portal — Dashboard, showing the vendor approval queue and approved vendors with reliability scores.

The demo data set (QuickMart and FreshStore, both grocery vendors with two products each in stock) is sufficient to exercise the routing and approval flows end to end, though testing so far has not yet stressed the re-routing and delivery-partner retry paths under load — this is flagged as the primary gap to close before the next evaluation.

Evaluation Metric	Target Specification	Achieved (Prototype)	Status
Order routing latency	≤ 2.0 s	[TBD]	[TBD]
Vendor match accuracy	$\geq 90.0\%$	[TBD]	[TBD]
Checkout success rate	$\geq 98.0\%$	[TBD]	[TBD]
API p95 response time	≤ 300 ms	[TBD]	[TBD]
Concurrent order throughput	20 orders/min	[TBD]	[TBD]
Real-time update delivery	≤ 1.0 s	[TBD]	[TBD]

Table 3.2: VendorFlow Prototype Performance Metrics Summary

Chapter 4

Conclusion

4.1 Summary of Work

During the first half of the project lifecycle, the team completed the data model at three levels of detail (conceptual, logical, and physical ER diagrams), the cross-functional order-processing activity diagram, and a functional full-stack prototype covering authentication and the customer, vendor, and admin portals. VendorFlow demonstrates role-based access control, multi-vendor product browsing, vendor inventory management, and an admin approval workflow with reliability scoring.

4.2 Key Findings

1. **Splitting by workload clarifies ownership:** Separating the core API (C++/Java), the routing service (Python), and the frontend (TypeScript) into distinct services kept each team member's work independently testable.
2. **Reliability scoring needs real order volume:** The accept/re-route logic is straightforward to implement, but its tuning will only be meaningful once there is enough order history to distinguish genuinely unreliable vendors from ones with temporarily low stock.
3. **Prescription verification is a manual bottleneck:** Routing pharmacy orders through a human admin for prescription verification is correct for v1, but will need a queueing/SLA mechanism as order volume grows.

4.3 Future Work and Recommendations

For the remainder of the project, the team plans to implement:

1. The delivery-partner and analytics-dashboard portals, which are implemented behind the existing authentication flow but not yet finalized.
2. Load testing of the routing and delivery-assignment retry paths to populate the performance metrics in Section 3.3 with real figures.
3. Payment gateway integration and an AI-assisted demand-forecasting module, as scoped for the v2 release.

4. Production deployment with Redis-backed rate limiting and structured audit logging.

4.4 Final Remarks

VendorFlow gives independent grocery and pharmacy vendors a shared, affordable way to sell online and coordinate delivery, while giving customers one place to discover and order from every approved local store nearby. The current prototype validates the core architecture and data model end to end; the next phase focuses on completing the remaining portals and hardening the system under realistic load.

Bibliography

- [1] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” *IETF RFC 7519*, 2015.
- [2] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [3] Redis Ltd., *Redis Documentation*. [Online]. Available: <https://redis.io/docs/>
- [4] Socket.IO, *Socket.IO Documentation*. [Online]. Available: <https://socket.io/docs/>